

Exercises: Model Predictive Control (MPC)

Exercise 1

In this exercise, you will use `MPCrun.m` or `MPCrun.py` to implement an MPC controller based on the optimal control functions for the planner robot that you defined in the previous lecture exercises (Exercise 2 and Exercise 3 from *Intro to Optimal Control: Lifting (Batch Optimization)*). The provided script supports both a **deterministic** run (perfect position measurement, no process noise) and a **stochastic** run (measurement noise and process noise).

- Ensure that your `.casadi` files for the optimal control problems are available. In addition, run `make_linear_kf_casadi.m/py` and `make_ekf_nonlinear_robot_dv_casadi.m/py` to generate Kalman filter `.casadi` functions for the linear and nonlinear models.

Note: you need to fill in the missing parts of the code in `make_linear_kf_casadi.m/py`.

- Start with the deterministic version by setting `use_stochastic` to `False`. For both the linear and nonlinear solver, experiment with:
 1. the prediction horizon H ,
 2. the cost weights Q and R (and terminal weight Q_T , if used),
 3. the limits on the input and velocity,
 4. the reference frequency (e.g., increase/decrease the circle speed).
- Then switch to the stochastic version by setting `use_stochastic` to `True`.
 1. You may observe that the solver occasionally fails or returns an illogical predicted trajectory. Why might this happen in an MPC loop under noise and estimation error?
 2. To handle potential infeasibilities, implement a version of your *linear* and *nonlinear* optimal control problems with slack variables (soft constraints).
 3. Experiment with different levels of process noise and measurement noise, and observe how the filter tuning (e.g., Q^{KF} , R^{KF}) affects the closed-loop behavior.

Exercise 2 (Extra):

Consider a single-product inventory system. Let the state be the inventory level $x(k) \in \mathbb{R}$ (positive = items in stock, negative = backlog), and let the input $u(k) \in \mathbb{R}$ be the replenishment order placed at time k . Demand $d(k)$ is an exogenous signal (known forecast over the horizon, or treated as constant).

A simple discrete-time model is:

$$x(k+1) = x(k) + u(k) - d(k).$$

You measure inventory (possibly noisy):

$$y(k) = x(k) + v(k).$$

Design an MPC controller that keeps the inventory near a desired target x_{ref} while penalizing ordering effort, but with *smooth asymmetric penalties* that can weight shortages more than overstock. For example, use a differentiable stage cost such as

$$\ell(x, u) = q (x - x_{\text{ref}})^2 + r u^2 + \rho \text{softplus}(\alpha (x_{\min} - x)),$$

where $\text{softplus}(s) = \log(1 + e^s)$ is smooth and convex, and the last term penalizes inventory falling below a safety stock $x_{\min}$ (shortages), more strongly as α increases.

Choose realistic bounds:

$$0 \leq u(k) \leq u_{\max}, \quad x(k) \leq x_{\max}.$$

(Optionally, allow backlog by setting no lower bound on $x(k)$, and let the cost handle it.)

- Implement the MPC optimization over a horizon H using CasADi with the assumption of perfect measurements (no measurement error).
- Choose a demand forecast $d(k)$ over the horizon (e.g., constant, sinusoidal, or piecewise).
- Compare a quadratic-only cost versus your smooth nonquadratic penalty (e.g., softplus or smooth absolute value).
- (Optional) Add process noise $w(k)$ to the dynamics and design a Kalman filter for $x(k+1) = x(k) + u(k) - d(k) + w(k)$, $y(k) = x(k) + v(k)$ and run certainty-equivalent MPC.